

```

// EMERGENCE OS 1 - Context Modifier
const modifier = (text) => {
  try {
    EmergenceEngine.init();
    const safeText = text || "";

    const sceneNames = EmergenceEngine.getSceneNames(safeText);
    state.emergence.sceneNames = sceneNames;

    let notes = "";
    if (!state.emergence.isCommandTurn) {
      // Prefer AI Dungeon's own action counter over a self-incremented one, so
      // an edited or retried action doesn't drift the turn count out of sync.
      const turnCount = EmergenceEngine.syncTurnCount();
      EmergenceEngine.syncSettings();
      // Single dispatcher: runs at most one lightweight maintenance task per
      // turn. Undercurrents need no slot here at all - they're purely passive,
      // updated directly from the story text in the Output hook.
      EmergenceEngine.runPeriodicMaintenance(turnCount, safeText);

      const profileNudge = EmergenceEngine.buildProfileNudge();
      const romanceNudge = EmergenceEngine.buildRomanceNudge();
      const securityNudge = EmergenceEngine.buildSecurityNudge();
      const reflectionNudge = EmergenceEngine.attemptAskReflection(turnCount, sce
      const npcNote = EmergenceEngine.buildNpcNote();
      // Priority-packed instead of concatenate-then-truncate. Profile-fill is
      // a direct response to an explicit player command (/card), so it ranks
      // first. Romance/Security nudges are one-shot with no retry - miss them
      // once and that moment's gone for good - so they rank above the routine
      // reflection/color-note items, which will simply try again later.
      notes = EmergenceEngine.packByPriority([
        { text: profileNudge, priority: 1 },
        { text: romanceNudge, priority: 2 },
        { text: securityNudge, priority: 2 },
        { text: reflectionNudge, priority: 3 },
        { text: npcNote, priority: 4 }
      ], 1200);
    }

    const directive = EmergenceEngine.buildDirective();

    if (!state.memory) state.memory = {};
    // frontMemory has no player-facing UI equivalent, so it's always safe to
    // set directly every turn.

```

```
state.memory.frontMemory = directive;

// authorsNote is DIFFERENT: AI Dungeon players can type their own custom
// Author's Note in the story's own settings. Setting this field to an
// empty string on quiet turns was assumed to gracefully fall back to
// whatever the player had typed there - confirmed via direct testing that
// it does NOT; it wipes the player's own note instead. So this only ever
// touches the field on turns where there's genuinely something to add,
// and leaves it completely alone otherwise, preserving whatever the
// player has configured on their own.
if (notes) {
    state.memory.authorsNote = notes;
}

return { text };
} catch (error) {
    return { text };
}
};
modifier(text);
```